

Red convolucional profunda para el Fashion-MNIST dataset: autoencoder y clasificador con y sin pre-entrenamiento

Diego Jesús Scacchi*

Facultad de Matemática, Astronomía, Física y Computación,
Universidad Nacional de Córdoba, Ciudad Universitaria, 5000 Córdoba, Argentina
(Dated: 2 de agosto de 2026)

En este trabajo se estudia la clasificación de prendas del conjunto de datos Fashion-MNIST mediante una arquitectura convolucional profunda que combina un autoencoder y un clasificador. Se evaluaron cuatro esquemas de entrenamiento del clasificador, diferenciados por el uso de pre-entrenamiento y la decisión de congelar o ajustar el codificador. Los resultados muestran que el pre-entrenamiento contribuye a mejorar la precisión, aunque puede acelerar el sobreajuste. El esquema seleccionado alcanza una precisión en validación de 92.3%. Finalmente, se discuten las implicancias de los distintos esquemas en la capacidad de generalización del modelo y posibles mejoras.

I. INTRODUCCIÓN

Las redes neuronales artificiales son un modelo de machine learning que pretende tomar decisiones imitando la manera en que las neuronas biológicas trabajan juntas para identificar fenómenos, sopesar opciones y llegar a conclusiones. Estos modelos computacionales permiten resolver problemas complejos de clasificación y regresión estableciendo conexiones entre las neuronas artificiales. La clasificación de imágenes se ha convertido en un área central dentro del aprendizaje profundo, donde las *redes convolucionales* han demostrado ser especialmente eficaces para capturar la estructura espacial de los datos. En este trabajo trataremos la clasificación de prendas del conjunto Fashion-MNIST [1] mediante una arquitectura que combina un autoencoder convolucional y un clasificador convolucional profundo. En conjunto con la librería PyTorch [2] es posible diseñar, entrenar y validar una red neuronal multicapa para clasificar estas imágenes. La inclusión del *autoencoder* permite realizar un pre-entrenamiento no supervisado, orientado a extraer representaciones compactas y generales de las imágenes, que luego se reutilizan como punto de partida para el clasificador. De esta manera, se busca analizar el impacto del pre-entrenamiento en la convergencia y la capacidad de generalización del modelo, comparando distintos esquemas de entrenamiento y evaluando su desempeño sobre el conjunto de validación. Esto nos permitirá a su vez determinar el valor de añadir un autoencoder a la red clasificadora para el dataset en cuestión.

II. TEORÍA

Las redes neuronales convolucionales (CNN) están compuestas por una sucesión de capas encargadas de extraer características, funciones de activación no lineales y, eventualmente, capas de reducción de resolución (*pooling*). Conforme aumenta la profundidad de la red, las primeras capas aprenden patrones simples (bordes, texturas y orientaciones), mientras que las capas profundas

representan estructuras cada vez más complejas y discriminativas. En este trabajo las CNN se utilizan tanto para construir un autoencoder convolucional como una red clasificadora, permitiendo reutilizar las representaciones aprendidas durante el pre-entrenamiento.

La operación fundamental de una CNN es la convolución discreta entre la imagen de entrada y un conjunto de filtros (o *kernels*) entrenables.

Cada filtro consiste en una matriz de pesos que se desplaza sobre la imagen produciendo un mapa de características o *feature map*.

Tres hiperparámetros determinan el comportamiento de una capa convolucional:

- **Kernel:** tamaño del filtro utilizado para recorrer la imagen. Kernels pequeños (por ejemplo 3×3) suelen capturar detalles locales con menor cantidad de parámetros.
- **Stride:** cantidad de píxeles que avanza el filtro en cada desplazamiento. Un stride mayor reduce la resolución espacial del mapa de activación.
- **Padding:** cantidad de píxeles agregados alrededor de la imagen antes de aplicar la convolución. Permite controlar la dimensión espacial de la salida y evitar la pérdida excesiva de información en los bordes.

El número de filtros determina la cantidad de características distintas que la red puede aprender en cada nivel de representación.

Las capas lineales (*fully connected* o *dense*) realizan una transformación afín de la forma

$$\mathbf{y} = W\mathbf{x} + \mathbf{b} \quad (1)$$

donde W representa la matriz de pesos y $\mathbf{b}$ el vector de sesgos.

Para este proyecto, las capas lineales permiten reducir la dimensión hasta obtener una representación compacta del espacio latente, aumentarla cuando requerimos de

una dimensión mayor (pos-compresión) y realizar la clasificación final sobre las diez categorías.

El *espacio latente* corresponde a una representación de baja dimensión capaz de conservar la información más relevante de los datos originales. En un autoencoder, el codificador transforma la imagen de entrada en un vector latente, mientras que el decodificador intenta reconstruir la imagen original únicamente a partir de dicha representación.

La calidad del espacio latente determina el equilibrio entre compresión y capacidad de reconstrucción. Cuando el codificador aprende representaciones suficientemente informativas, estas pueden reutilizarse posteriormente como extractor de características para tareas de clasificación mediante aprendizaje por transferencia (*transfer learning*).

Las funciones de activación introducen no linealidad en la red neuronal, permitiendo aproximar funciones altamente complejas. La función *Rectified Linear Unit* (ReLU) está definida por

$$\text{ReLU}(x) = \max(0, x). \quad (2)$$

Esta función introduce no linealidad en el modelo, permitiendo que la red aprenda representaciones complejas y mitigando el problema de la supresión del gradiente.

La función sigmoide está dada por

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (3)$$

Su salida pertenece al intervalo $(0, 1)$. En el presente trabajo se utiliza en la última capa del decodificador para garantizar que los valores reconstruidos permanezcan dentro del mismo rango de normalización efectuado sobre el Fashion-MNIST dataset.

Las funciones de pérdida cuantifican el error cometido por una red durante el entrenamiento y constituyen la magnitud que el algoritmo de optimización intenta minimizar.

El Error Cuadrático Medio (MSE) se define como

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2. \quad (4)$$

Esta función penaliza cuadráticamente las diferencias entre la imagen original y la reconstruida. Dado que el objetivo del autoencoder consiste precisamente en reconstruir la entrada, esta elección resulta adecuada ya que nos permite minimizar el error pixel a pixel.

Para problemas de clasificación multiclase, se utiliza la *entropía cruzada* (*Cross-Entropy Loss*), expresada como:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{i,j} \log(\hat{y}_{i,j}) \quad (5)$$

donde N es el número total de ejemplos, C el número de clases, $y_{i,j}$ indica si el ejemplo i pertenece a la clase j , y $\hat{y}_{i,j}$ es la probabilidad predicha para esa clase. Esta función mide el error de clasificación y orienta el proceso de ajuste de los parámetros.

El conjunto de datos a analizar contiene un total de 70 000 imágenes en escala de grises, de las cuales 60 000 se utilizan para entrenamiento y 10 000 para validación o prueba. El uso diferenciado de los conjuntos de entrenamiento y validación resulta fundamental para medir el desempeño real del modelo. Mientras que el conjunto de entrenamiento permite ajustar los parámetros internos de la red neuronal, el conjunto de validación ofrece una evaluación independiente que ayuda a detectar sobreajuste y a estimar la capacidad de generalización del modelo frente a datos no vistos. Cada imagen representa una prenda de vestir perteneciente a una de diez categorías: remera/top, pantalón, pulóver, vestido, abrigo, sandalia, camisa, zapatilla, bolso y bota. En la Figura 1 se ve un ejemplo de una muestra aleatoria de imágenes del conjunto de datos con sus respectivas etiquetas. En este trabajo, las imágenes fueron normalizadas y transformadas en tensores antes de ser utilizadas como entradas de la red neuronal.

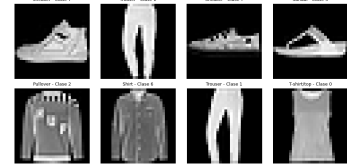


Figura 1: Ejemplo de imágenes aleatorias del conjunto de datos Fashion-MNIST con sus respectivas etiquetas.

III. MÉTODOS

A. Arquitectura del Autoencoder

El autoencoder implementado posee una arquitectura simétrica compuesta por un **codificador** convolucional, un cuello de botella completamente conectado y un **de-codificador** basado en convoluciones transpuestas. El codificador recibe imágenes de tamaño $1 \times 28 \times 28$ y extrae progresivamente características mediante dos capas convolucionales con activación *ReLU*, seguidas de una operación de *max-pooling* que reduce la resolución espacial y de una capa *dropout* para mejorar la capacidad de generalización del modelo. La representación obtenida se aplanan y, opcionalmente, se proyecta a un espacio latente de dimensión $n = 128$ mediante una capa completamente conectada, constituyendo el *cuello de botella* del autoencoder. Posteriormente, una segunda capa lineal reconstruye la representación convolucional original, que es procesada por el decodificador. Este último invierte

el proceso de compresión mediante dos capas de convolución transpuesta, utilizando activaciones ReLU en las capas intermedias y una función Sigmoid en la salida para restringir los valores reconstruidos al intervalo $[0, 1]$.

B. Arquitectura del Clasificador

La red clasificadora reutiliza el codificador previamente entrenado del autoencoder como extractor de características. Sobre dichas características se construye un clasificador formado por dos capas lineales ocultas de tamaños $n_1 = 128$ y $n_2 = 64$, ambas seguidas de funciones de activación ReLU y capas de *dropout*, finalizando con una capa lineal de diez neuronas correspondiente a las clases del conjunto Fashion-MNIST. Durante la etapa de pre-entrenamiento únicamente se optimizan los parámetros del clasificador, manteniendo congelados los pesos del codificador convolucional, mientras que en los experimentos posteriores se comparan distintos esquemas de entrenamiento para evaluar el impacto del pre-entrenamiento sobre el desempeño final del modelo.

IV. RESULTADOS

A continuación presentamos la serie de experimentos realizados con las distintas arquitecturas, con la meta de encontrar: el autoencoder convolucional óptimo, el clasificador convolucional de mejor rendimiento (utilizando como módulo codificador lo aprendido por el autoencoder), y finalmente el esquema que nos permitirá decidir sobre la efectividad del pre-entrenamiento.

A. Autoencoder convolucional

Con el fin de obtener el autoencoder de mejor performance, se realizaron un total de 5 experiencias, partiendo de un modelo *baseline* (modelo 0). Este último posee un espacio latente de 128 neuronas, dropout en el módulo codificador de $p_1 = 0.2$, filtros $[8, 16]$, la presencia del cuello de botella activa, stride $[1, 1, 2, 1]$ y padding $[0, 0, 0, 0]$. El optimizador utilizado fue *Adam*, con un learning rate $\eta = 10^{-3}$.

Una serie de ejecuciones preliminares permitieron determinar que una cantidad de 30 épocas resulta adecuada para efectuar la comparación de los experimentos.

La Tabla I exhibe las variaciones efectuadas y los resultados obtenidos en cada experiencia. Cabe mencionar que, para el experimento 6, se agregaron dos capas lineales con activación *ReLU* en el cuello de botella que comprimen/descomprimen a una dimensión de 256. Como la compresión al espacio latente quedó entre medio de las mismas, nos permite observar el fenómeno de *progressive compressing* para este dataset.

Podemos deducir que las modificaciones de los modelos 1, 2, 3 y 5 resultaron beneficiosas respecto al modelo base, puesto que los errores promedio obtenidos tanto en el entrenamiento como en testeo son menores. Por otro lado, aumentar la tasa de aprendizaje o acomplejar la arquitectura del cuello de botella resultan contraproducentes. Se seleccionará entonces el modelo 3 (sin presencia de bottleneck) para continuar con las demás secciones. Esto nos da la pauta de que, para el Fashion-MNIST dataset, no es necesaria la compresión de los datos a un espacio latente.

B. Clasificador convolucional profundo con pre-entrenamiento

Para esta instancia, utilizaremos el autoencoder óptimo como módulo de entrada para el clasificador y entrenaremos *solo* los parámetros de la red clasificadora. Se propone un conjunto de 3 experimentos, partiendo de un clasificador *base*.

Los hiperparámetros baseline son: capas ocultas de $n_1 = 128$ y $n_2 = 64$ neuronas respectivamente, tasa de aprendizaje de 10^{-3} , optimizador *Adam* y dropout $p = 0.2$. Las variaciones realizadas son:

- Modelo 1: $n_1 = 512$ y $n_2 = 256$
- Modelo 2: optimizador *SGD*
- Modelo 3: $p = 0.4$

Lo presentado en la Tabla II nos permite concluir que el modelo baseline es el candidato a ser comparado con el resto de los esquemas para evaluar la importancia del pre-entrenamiento. Incluso aumentando el número de neuronas en las capas ocultas (modelo 1), lo cual permite aprender mayor cantidad de patrones, resulta innecesario al momento de clasificar las imágenes de prendas de ropa.

Dado que calculamos la precisión de la red clasificadora tanto en el entrenamiento como en el conjunto de validación, podemos entonces determinar la época a partir del cual el clasificador no aumenta su capacidad de generalización. Esto nos permitirá realizar simulaciones posteriores sin costo computacional innecesario.

Modelo	Error prom. en train	Error prom. en val	Variación de hiperparámetro
0	0.007387	0.007570	Baseline
1	0.005303	0.005714	Cantidad de filtros (8,16) $\rightarrow$ (32,64)
2	0.006451	0.006656	Kernels (5,5,2,5), Strides (1,1,0,0), Padding (2,2,1,1)
3	0.004959	0.004993	Sin cuello de botella (BN)
4	0.008965	0.009072	Tasa de aprendizaje 10^{-4}
5	0.007046	0.007220	Dropout $p_1 = 0.4$
6	0.007724	0.007975	Añadir capas lineales en el bottleneck

Tabla I: Resultados de los experimentos con el autoencoder tras 30 épocas de entrenamiento. Se presentan tanto las variaciones en la configuración de hiperparámetros, como las métricas de desempeño resultantes.

Modelo	Prec. Train	Prec. Val	Pérdida Train	Pérdida Val
0	90.1 %	88.2 %	0.262816	0.316525
1	89.7 %	87.6 %	0.264945	0.333528
2	57.6 %	57.3 %	1.139366	1.144555
3	88.3 %	87.0 %	0.313260	0.352834

Tabla II: Resultados promedio de precisión y pérdida para los distintos modelos de clasificador. En todos los casos, se iteró sobre un total de 30 épocas.

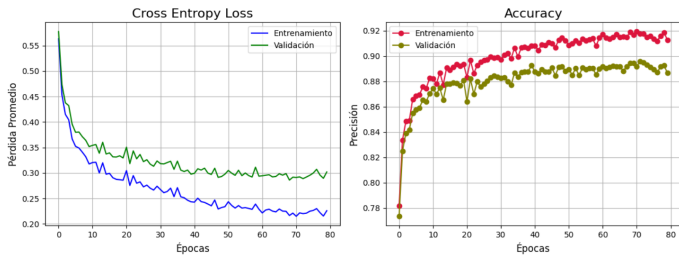


Figura 2: Pérdida y precisión de los conjuntos de entrenamiento y validación para la red convolucional profunda baseline (modelo 0). Se itera hasta un máximo de 80 épocas.

De la Figura 2 podemos observar que no hay mejora significativa en las métricas del conjunto de validación desde la época 40. Por lo tanto, el resto de los experimentos utilizarán esa cantidad como máximo.

C. Efectividad del pre-entrenamiento

Para poder responder a esta premisa, se propusieron 4 esquemas diferentes de redes clasificadoras. En cada uno de ellos, se alterna entre utilizar lo aprendido por el autoencoder como módulo de entrada o inicializar los pesos aleatoriamente, y decidir si continuar entrenando los hiperparámetros del autoencoder o no durante la ejecución del clasificador.

En la Tabla III se muestran los resultados obtenidos

en términos de pérdida y precisión en el conjunto de entrenamiento y validación. Se incluyen además las características de cada uno de los esquemas.

Considerando lo representado en la Figura 4 y la tabla mencionada anteriormente, podemos concluir que el esquema con peor rendimiento es cuando se entrenan todos los componentes partiendo de pesos aleatorios. Es claro que, a pesar de que se logra una alta precisión en entrenamiento y validación, hay una tendencia de la red por aprender patrones específicos, mostrando un marcado *overfitting*. Por otra parte, el esquema baseline logra una buena capacidad de generalización, a expensas de no lograr la máxima precisión de los 4 esquemas. El congelamiento del autoencoder aleatorio (esquema 3) produce una mejora en el overfitting ya que la restricción impuesta no le permite seguir agregando ruido o rasgos característicos del dataset. Finalmente, podemos decir que el esquema 2 (fine-tuning) es el más balanceado en términos de lograr una buena precisión en ambos conjuntos, sin comprometer la capacidad de generalización.

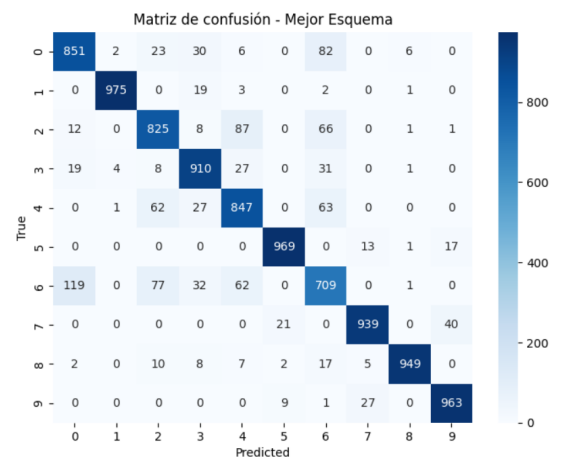


Figura 3: Matriz de confusión del esquema seleccionado. Esquema 2.

Esquema	Prec. Train	Prec. Val	Pérdida Train	Pérdida Val	Descripción
0	89.7 %	87.6 %	0.274087	0.329819	Pretrain + autoencoder congelado
1	98.9 %	92.2 %	0.037212	0.264101	Entrenamiento completo aleatorio
2	92.3 %	89.6 %	0.208589	0.289464	Fine-tuning. Pretrain + complete train
3	92.5 %	89.4 %	0.201377	0.297573	Autoencoder aleatorio congelado + entr. del clasificador

Tabla III: Resultados promedio de precisión y pérdida para los distintos esquemas de entrenamiento del clasificador.

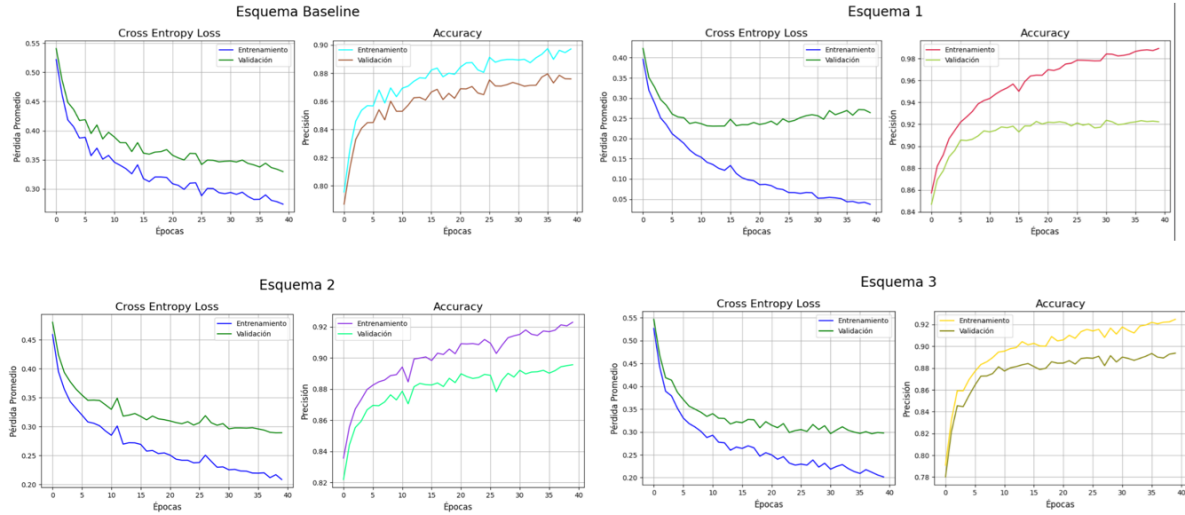


Figura 4: Pérdida y precisión para los 4 esquemas de red clasificadora convolucional profunda. En todos los casos se iteró sobre un total de 40 épocas.

V. CONCLUSIONES

Los gráficos resultantes muestran que las redes convolucionales profundas multicapa de los cuatro esquemas analizados logran clasificar precisamente las imágenes provistas [1], obteniendo una precisión en el conjunto de validación mayor o igual al 88 %.

Lo obtenido para el esquema 2 nos permite concluir que en ciertas ocasiones el autoencoder logra proporcionar información relevante que puede ser reutilizada por el clasificador, produciendo así transferencia de aprendizaje. La matriz de confusión nos muestra que hubo una gran cantidad de aciertos para cada clase, y las imágenes que se encuentran fuera de la diagonal son despreciables en comparación con las etiquetas correctas.

Otras posibles configuraciones a experimentar a futuro podrían ser considerando combinaciones cruzadas de hiperparámetros (aumentar la dimensión en las capas ocul-

tas, pero reduciendo el espacio latente y agregando dropout).

En conjunto, las variantes exploradas ponen de manifiesto la importancia de experimentar con diferentes configuraciones de la red neuronal en función del dataset a abordar y el objetivo que se quiere alcanzar.

* diego.scacchi@mi.unc.edu.ar

- [1] H. Xiao, K. Rasul, and R. Vollgraf, Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms, arXiv preprint arXiv:1708.07747 (2017).
- [2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, Pytorch: An imperative style, high-performance deep learning library, in *Advances in Neural Information Processing Systems (NeurIPS 2019)* (2019).